GitHub: Conceitos Essenciais e Colinha de Comandos

Guia do Participante — Oficina de GitHub

Projeto de Extensão II — Sistemas de Informação (IFSP)

---

1. Por que versionar código?

Quem já trabalhou em grupo conhece o problema: `trabalho_final.docx`, `trabalho_final_v2.docx`, `trabalho_final_v2_AGORA_VAI.docx`. Ninguém sabe qual é a versão certa, quem mexeu no quê, nem como voltar ao que funcionava ontem.

Controle de versão resolve isso. É um sistema que registra cada alteração feita nos arquivos de um projeto, guardando **o que mudou, quem mudou e quando**.

O que se ganha com isso:

- **Histórico:** é possível voltar a qualquer ponto anterior do projeto.
- **Segurança para experimentar:** testar uma ideia nova sem estragar o que já funciona.
- **Colaboração:** várias pessoas trabalham no mesmo projeto sem sobrescrever o trabalho umas das outras.
- **Backup:** o projeto fica guardado também na nuvem.
- **Portfólio:** o GitHub funciona como vitrine pública do que você produz, e recrutadores costumam olhar esses perfis.

---

2. Git x GitHub

São duas coisas diferentes que trabalham juntas.

	Git	**GitHub**
O que é	Ferramenta de controle de versão	Plataforma online que hospeda repositórios Git
Onde roda	No seu computador	Na nuvem (site)
Para que serve	Registrar o histórico do projeto	Guardar, compartilhar e colaborar em projetos
Precisa de internet?	Não (funciona offline)	Sim
Recursos extras	Branches, commits, merges	Pull Requests, Issues, Actions, Pages, perfil público

Pontos importantes:

- O Git foi criado em 2005 por Linus Torvalds, o criador do Linux. O GitHub surgiu depois, como uma plataforma para hospedar projetos Git, e hoje pertence à Microsoft.
- O GitHub não é a única opção: existem alternativas como GitLab e Bitbucket. Todas usam o Git por baixo, então o que você aprende vale para qualquer uma delas.

---

3. Como o Git “pensa”

3.1 Snapshots (fotos do projeto)

O Git não guarda só “o que mudou”. A cada commit, ele registra uma **fotografia completa** do estado do projeto naquele momento. Por isso é possível voltar a qualquer ponto da história com precisão.

3.2 As três áreas de trabalho

Todo arquivo passa por três áreas antes de ir para o GitHub:

Área	**O que é**
Pasta de trabalho (working directory)	Onde você edita os arquivos normalmente
Staging (área de preparação)	A “fila” dos arquivos que você escolheu para entrarem no próximo commit
Repositório local	O histórico de commits guardado no seu computador

Por que existe o staging? Para você escolher **exatamente o que entra** em cada commit. Se você alterou 5 arquivos mas só 2 têm a ver com a mesma ideia, commita só esses 2 e mantém o histórico organizado.

3.3 Onde as coisas ficam

```
Pasta de trabalho ──git add──► Staging ──git commit──► Repo local ──git push──► GitHub
                                                                  ◄──── git pull ────
```

> *Atenção: `add` e `commit` acontecem **só no seu computador**. O GitHub só muda depois do `push`. Muita confusão de iniciante vem de achar que o commit já foi para a nuvem.*

3.4 Anatomia de um commit

Cada commit guarda:

- **Um identificador único** (o *hash*), uma sequência longa de letras e números, como `a3f9c2e...`
- **O autor** (nome e e-mail configurados no Git)
- **A data**
- **A mensagem**, que explica o que foi feito
- **Uma referência ao commit anterior**, e é isso que forma a "corrente" do histórico

---

4. Branches: universos paralelos

Branch é uma linha paralela de desenvolvimento. Ela permite trabalhar numa ideia nova sem mexer na versão principal do projeto.

- A branch principal costuma se chamar **`main`** e deve conter a versão estável.
- Para cada nova funcionalidade ou correção, cria-se uma branch própria.
- Quando o trabalho está pronto e revisado, a branch é **mesclada** (*merge*) de volta na `main`.

```
main:          A ── B ── C ──────────── F
                          \            /
nova-branch:               D ── E ────
```

Nesse desenho, a `nova-branch` nasceu do commit C, recebeu dois commits (D e E) e voltou para a `main` por meio do merge (F).

HEAD é o "ponteiro" que indica **onde você está agora**: em qual branch e em qual commit.

Conflito de merge

Quando duas pessoas alteram **a mesma parte** do mesmo arquivo em branches diferentes, o Git não consegue decidir sozinho qual versão manter. Isso é um **conflito**.

Conflito **não é um erro grave**: é o Git pedindo que um humano decida. O arquivo é marcado com `<<<<<<<`, `=======` e `>>>>>>>`, mostrando as duas versões. Você escolhe o texto final, apaga as marcações e conclui o merge.

---

5. Trabalhando com repositórios remotos

- **Remote / origin:** a cópia do repositório que fica no GitHub. `origin` é o nome padrão dado a ela.
- **Clone:** baixa uma cópia **completa** do repositório (arquivos e todo o histórico) para o seu computador.
- **Push:** envia seus commits locais para o GitHub.
- **Pull:** traz para o seu computador o que mudou no GitHub, para você ficar atualizado.
- **Fork:** cria uma **cópia do repositório de outra pessoa na sua conta**. É o jeito de contribuir com projetos dos quais você não é membro: você altera a sua cópia e propõe a mudança ao projeto original.

---

6. Pull Request e revisão de código

Pull Request (PR) é um pedido para que as alterações de uma branch sejam mescladas em outra, normalmente na `main`.

Mais do que um botão de merge, o PR é um **espaço de conversa**:

- Mostra exatamente **o que mudou** (aba *Files changed*).
- Permite que outras pessoas **comentem linha por linha**, sugiram melhorias e aprovem.
- Registra a **discussão** e o motivo das decisões.
- Pode rodar **verificações automáticas** (testes) antes de aceitar.

Em equipes de desenvolvimento, é comum que **ninguém envie código direto para a `main`**: tudo passa por Pull Request.

---

7. Conhecendo um repositório no GitHub

As abas principais da página de um repositório:

Aba	Para que serve
Code	Os arquivos do projeto, branches e o histórico de commits
Issues	Lista de tarefas, bugs e ideias do projeto
Pull requests	Pedidos de mesclagem em andamento
Actions	Automações (por exemplo, rodar testes a cada envio)
Projects	Quadro de organização de tarefas (estilo kanban)
Insights	Estatísticas e atividade do repositório
Settings	Configurações, colaboradores e permissões

Arquivos que costumam existir em um repositório:

- **README.md:** o "cartão de visitas" do projeto. Explica o que ele faz e como usar. É a primeira coisa que as pessoas leem.
- **.gitignore:** lista o que o Git deve **ignorar** (arquivos temporários, pastas de dependências, senhas e chaves).
- **LICENSE:** define como outras pessoas podem usar o seu código.

Outros recursos úteis: o **GitHub Pages** publica sites estáticos direto de um repositório, e o **perfil** (e o README do perfil) funciona como seu portfólio.

---

8. O fluxo de trabalho em equipe

O ciclo mais usado, em resumo:

1. **Atualizar** o projeto local (`pull`)
2. **Criar uma branch** para a tarefa
3. **Fazer alterações** e salvar em commits pequenos, com boas mensagens
4. **Enviar** a branch para o GitHub (`push`)
5. **Abrir um Pull Request** e receber revisão
6. **Mesclar** na `main` e apagar a branch usada

---

9. Boas práticas

- **Commits pequenos e frequentes:** cada commit deve representar **uma ideia** só.
- **Mensagens claras:** curtas, no imperativo, dizendo **o que** mudou. Bom: `Corrige link do menu`. Ruim: `mudanças`, `ajustes`, `asdf`.
- **Uma branch por tarefa**, com nome que explique o objetivo (por exemplo, `adiciona-pagina-contato`).
- **Não trabalhe direto na `main`** em projetos de equipe.
- **Atualize antes de começar** (`pull`) para evitar conflitos.
- **Nunca envie segredos** (senhas, tokens, chaves de API). O que sobe para o GitHub fica no histórico, mesmo depois de apagado. Se vazar um segredo, **revogue-o imediatamente**.
- **Proteja a conta:** ative a verificação em duas etapas (2FA) e use um e-mail que você acessa.
- **O GitHub não aceita a senha da conta** para enviar código pelo terminal. A autenticação é feita pelo Git Credential Manager (login pelo navegador), por token de acesso pessoal ou por chave SSH.

---

10. Vocabulário essencial

Termo	Significado simples
Repositório	A pasta do projeto + todo o histórico dela
Commit	Um ponto salvo na história, com uma mensagem (como uma "foto" do projeto)
Hash	O identificador único de cada commit
Staging	A "fila" dos arquivos escolhidos para o próximo commit
Branch	Uma linha paralela de trabalho (a principal costuma se chamar `main`)
HEAD	O ponteiro que indica onde você está (branch e commit atuais)
Merge	Juntar o trabalho de uma branch em outra
Conflito	Quando o Git não consegue juntar sozinho duas alterações na mesma parte de um arquivo
Remote / origin	A cópia do repositório que fica no GitHub
Clone	Baixar uma cópia completa de um repositório
Fork	Copiar o repositório de outra pessoa para a sua conta
Push	Enviar seus commits para o GitHub

Pull	Trazer para o seu computador o que mudou no GitHub
Pull Request (PR)	Pedido para juntar uma branch na principal, com espaço para revisão
Issue	Um registro de tarefa, bug ou ideia dentro do repositório
README	O arquivo de apresentação do projeto
.gitignore	O arquivo que lista o que o Git deve ignorar

11. Colinha de comandos

Para quê	**Comando**
Ver a versão do Git	`git --version`
Configurar nome / e-mail	`git config --global user.name "Nome"` / `git config --global user.email "email"`
Iniciar repositório na pasta atual	`git init`
Clonar um repositório	`git clone URL`
Ver o que mudou	`git status`
Ver diferenças ainda não commitadas	`git diff`
Adicionar um arquivo / tudo	`git add arquivo` / `git add .`
Salvar um commit	`git commit -m "mensagem"`
Ver histórico resumido	`git log --oneline`
Ver histórico em forma de árvore	`git log --oneline --graph --all`
Enviar para o GitHub	`git push`
Trazer novidades do GitHub	`git pull`
Ver o endereço do remoto	`git remote -v`
Listar branches	`git branch`
Criar e entrar numa branch	`git switch -c nome`
Trocar de branch	`git switch nome`
Juntar uma branch na atual	`git merge nome`
Apagar uma branch já mesclada	`git branch -d nome`
Tirar arquivo da fila (staging)	`git restore --staged arquivo`
Descartar alterações não commitadas	`git restore arquivo` (**não tem volta**)
Guardar trabalho pela metade	`git stash` / `git stash pop`